From Langium Grammars to Blockly Editors A Restricted-Subset Code Generation Pipeline

Generative Software Engineering

Md Mehedi Hasan and Julian Klüber

Bauhaus-Universität Weimar

Abstract. Textual grammars are an efficient way to define domain-specific languages (DSLs), but they require users to learn concrete syntax before they can write anything valid. Visual, block-based editors such as Google's Blockly lower this barrier by letting users assemble programs from interlocking blocks instead of typing text, at the cost of someone having to hand-build a block definition and code generator for every DSL.

We present a pipeline that closes this gap automatically: given a Langium grammar restricted to a well-defined subset of constructs, our tool validates it, builds a small intermediate representation (IR) capturing what each grammar rule *means* as a user-facing input, and emits a ready-to-run Blockly block editor — block definitions, a code generator that reconstructs the DSL's own concrete syntax, and the wiring to run it in a browser. We describe the pipeline's four stages, two extensibility case studies we carried out ourselves (cross-references rendered as a live, workspace-scanning dropdown field, and Langium's unordered-group operator `&`), and validate the implementation against a 45-test unit/integration suite, a 12-test coverage suite, and a performance benchmark across synthetic grammars of up to 200 rules.

1 Introduction

1.1 Background

Domain-specific languages let subject-matter experts express solutions in vocabulary close to their problem, rather than in a general-purpose programming language. Building a DSL from a textual grammar is well supported by modern language workbenches: Langium [2] is an open-source TypeScript framework for implementing DSLs, generating a parser, an abstract syntax tree (AST), and a Language Server Protocol implementation directly from an `.langium` grammar file.

Textual syntax is not the only way to interact with a DSL, however. Block-based visual programming environments, popularised by Scratch and, in library form, by Google's Blockly [1], let users construct programs by dragging and connecting graphical blocks. Blocks encode syntax structurally (a block that plugs into a statement socket *is* syntactically valid there by construction), which

removes an entire class of errors a textual editor would otherwise have to catch after the fact, and makes DSLs approachable to users who are not yet comfortable typing code by hand.

1.2 Motivation

Building a Blockly editor for a DSL by hand means writing one block definition and one code-generator function per grammar rule, and keeping both in sync with the grammar as it evolves — tedious, repetitive, and easy to let drift out of sync. Since a Langium grammar already declares, precisely, what each rule's concrete syntax looks like (its sequence of keywords, assigned features, and sub-rule references), most of the information needed to derive a corresponding block definition is already present in the grammar itself.

This motivates the project's central question: for a well-defined, restricted subset of Langium grammars, can a block editor — definitions, generator, and workspace — be derived automatically, rather than written by hand for every DSL?

2 Problem Statement

Given a Langium grammar G whose parser rules use only constructs from a supported subset S (Section 4.6 defines S precisely), the pipeline must produce:

block definitions one Blockly block type per parser rule in G, whose input fields/sockets correspond to that rule's assigned features;

a code generator a function per block type that reconstructs a syntactically valid instance of G's concrete syntax from the values a user has entered into that block's fields and connected inputs;

a runnable editor a Blockly workspace, pre-populated with a toolbox containing every generated block, that re-renders the generated DSL text live as the user edits the workspace.

If G uses a construct outside S, the pipeline must reject it before generating anything, reporting every offending construct and the rule it occurs in — not merely the first one found.

3 Example & Intuition

Consider the bundled `Recipe` grammar (Listing 1.1): a `Cookbook` names itself and contains zero-or-more `Ingredient`s followed by one-or-more `Step`s, and each `Step` names which previously declared `Ingredient` it uses via a Langium *cross-reference* (`ingredient=[Ingredient]`) rather than nesting a new one.

```
1 grammar Recipe
2
3 entry Cookbook:
```

```
    'recipe' name=ID '{'
        ingredients+=Ingredient*
        steps+=Step+
    '}';

Ingredient:
    'ingredient' name=ID amount=INT;

Step:
    'step' number=INT action=('add' | 'mix' | 'cook' | 'serve
    ')
    ingredient=[Ingredient];
```

Listing 1.1. The bundled `recipe.langium` example grammar

Running `node generate_blockly/src/parse.js generate_blockly/input/recipe.langium` turns this into three blocks. The `Step` block's generated Blockly JSON (Listing 1.2) shows the interesting case: `ingredient=[Ingredient]` becomes a `field_reference` — a custom dropdown field (Section 4.3) rather than a plain text box, since `Ingredient` declares a plain `name=ID` field the dropdown can scan for.

```
{
  "type": "step",
  "message0": "step Number: %1 Action: %2 Ingredient: %3",
  "args0": [
    { "type": "field_number",   "name": "NUMBER" },
    { "type": "field_dropdown", "name": "ACTION",
      "options": [["add","add"],["mix","mix"],
                  ["cook","cook"],["serve","serve"]] },
    { "type": "field_reference", "name": "INGREDIENT",
      "referencesType": "ingredient", "nameField": "NAME" }
  ],
  "previousStatement": "step", "nextStatement": "step"
}
```

Listing 1.2. Generated block JSON for the `Step` rule (excerpt)

Assembling one `cookbook` block, one `ingredient` block named `flour`, and one `step` block that picks `flour` from its (live-scanned) dropdown, the generator reconstructs the following DSL text, character-for-character valid `Recipe` syntax:

```
recipe pancakes {
  ingredient flour 200
  step 1 mix flour
}
```

Listing 1.3. Reconstructed DSL text for the workspace described above

This round-trip — grammar in, working editor out, edited workspace back to valid concrete syntax — is exactly what every test in Section 5.1 exercises for the pipeline's other bundled grammars.

4 Implementation

4.1 Pipeline Architecture

The pipeline (`generate_blockly/src/`) is organised as four sequential stages, each with a single, narrow responsibility:

1. `grammar-loader.js` parses the `.langium` file using Langium's own grammar services (`createLangiumGrammarServices`) and throws if the document has any error-severity diagnostic — a real syntax or linking error, not merely an unused-rule warning.
2. `validator.js` walks every parser rule's definition tree and collects — rather than fails fast on — every AST node type or cardinality that falls outside the supported subset, so a user gets one complete report instead of fixing violations one at a time.
3. `ir-builder.js` walks the (now validated) AST and produces an array of `RuleIR` objects: a rule name plus an ordered list of `IRPart`s, the pipeline's own grammar-agnostic vocabulary for "what does this piece of the rule mean as a block input."
4. `blockly-ts-target.js` turns `RuleIR[]` into three TypeScript files — `blocks.ts` (block definitions), `generator.ts` (the `forBlock` functions that reconstruct DSL text), and `main.ts` (workspace + toolbox wiring) — which are written directly into `blockly_app/src/`, ready for `npm run dev` to serve.

Separating stage 3 (grammar semantics) from stage 4 (Blockly-specific code generation) behind the small `IRPart` vocabulary is what let us extend the pipeline (Sections 4.3 and 4.4) by touching only one or two of the four stages at a time, rather than the whole codebase.

4.2 The Intermediate Representation

Every `IRPart` carries a `kind` drawn from six possibilities, summarised in Listing 1.4. A `feature=ID`/`feature=INT` assignment becomes a `field` (a text or number input); `feature=SomeRule` becomes a `value` (a plug-in socket); `feature+=Rule` becomes a `statement` (a stacking socket, since Blockly's native idiom for repetition is stacking one block per list item); an all-keyword `Alternatives` becomes a `dropdown`; and `feature=[Rule:TERMINAL]` — a Langium cross-reference — becomes a `reference` (Section 4.3).

```
/**
 * @typedef {Object} IRPart
 * @property {"keyword"|"field"|"dropdown"|"value"
 *           |"statement"|"reference"} kind
 * @property {string}   [feature]     - grammar feature name
 * @property {string}   [refRuleName] - referenced rule, if
   any
 * @property {boolean}  [optional]    - cardinality "?"
```

```
 * @property {boolean}  [repeatable]  - cardinality
    "*"/"+"/"+="
 */
```

Listing 1.4. The `IRPart` shape (JSDoc typedef, abbreviated)

A dedicated `nodeHandlers` registry (keyed by Langium AST `$type`) dispatches the traversal, so supporting a new grammar construct means adding one handler, not editing existing ones — the pattern both of our own extensions below follow.

4.3 Cross-References as a Live-Scanning Dropdown

A Langium cross-reference (`assignee=[Member:ID]`) means "point at an already-declared `Member` by name," as opposed to `assignee=Member`, which nests a brand-new one. Blockly has no built-in widget for this — a plain `field_dropdown`'s option list is fixed at block-definition time, while a cross-reference's valid targets change as the user edits the workspace. We therefore ship `FieldReference`, a hand-written `Blockly.FieldDropdown` subclass (`blockly_app/src/reference-field.ts`, the one file in `blockly_app/src/` *not* regenerated by the pipeline) that overrides `getOptions()` to scan `workspace.getAllBlocks()` for every block of the referenced type and read each one's declared name, live, every time the dropdown opens.

Wiring this up touches four files end to end: `ir-builder.js` records which rule a reference targets (and, via `computeNameFields`, which feature on *that* rule counts as its "name" — by convention, its first plain `feature=ID` field); `block-json-generator.js`'s `argBuilders.reference` emits {`"type": "field_reference", "referencesType": ..., "nameField": ...`}, falling back to a plain text field when the target rule has no name field to scan; `reference-field.ts` implements the field itself; and the generator in `blockly-ts-target.js` reads it back with the same `block.getFieldValue(...)` call used for any other field, unaware that the value came from a live dropdown rather than typed text. This trade-off is deliberate and documented: every declared name of the target type is offered regardless of where the referencing block sits on the workspace (Langium's own cross-references support finer-grained scoping we do not model), and a renamed or deleted target simply stops matching, falling back silently to the first available option.

4.4 Extending the Pipeline: Unordered Groups

As a second, independent extensibility case study, we added support for Langium's `UnorderedGroup` construct (`a & b & c`: all of these, in any order), following the same three-step pattern the codebase already documents for extending it. First, `validator.js` adds `'UnorderedGroup'` to its allowed-type set and walks its `.elements` the same way it already walks a `Group`'s. Second, `ir-builder.js` gains a new `nodeHandlers.UnorderedGroup` entry (Listing 1.5) whose body is, deliberately, an exact copy of the existing `Group` handler.

```
UnorderedGroup(node, ctx) {
    for (const e of node.elements)
        visit(e, ctx);
},
```

Listing 1.5. The `UnorderedGroup` IR handler

The design rationale is what makes this extension worth reporting rather than a one-line diff: Blockly's block UI is a static form — a block's fields render in one fixed order, decided once at block-definition time — so there is no way to preserve `&`'s "any order is fine" flexibility inside a single block. Committing to one fixed order (the grammar's own declaration order) is still always *correct*, however: since the grammar rule itself does not care what order its parts appear in, always emitting them in one particular order is trivially one of the orders the grammar accepts. What is lost is only the *choice* a person typing the DSL by hand would have had, not the validity of what the block editor produces. Because no new `IRPart.kind` was introduced, stage 4 (`blockly-ts-target.js`) required no changes at all — the clearest demonstration that separating IR construction from code generation (Section 4.1) pays off when extending the pipeline.

4.5 Tool Usage

The pipeline is driven from the command line:

```
npm install
node generate_blockly/src/parse.js generate_blockly/input/
    recipe.langium
npm run dev
```

Listing 1.6. Generating and running an editor for a grammar

The first command generates `blocks.ts/generator.ts/main.ts` into `blockly_app/src/`, overwriting the previous grammar's output in place; the second serves the resulting Blockly workspace via Vite.

4.6 Limitations

The validator's supported subset S is: `Group`, `UnorderedGroup`, `Alternatives`, `Assignment` (`=`/`+=`), `Keyword`, `RuleCall`, `CrossReference`, and cardinalities `?`/`*`/`+`/none. `Action` (Langium's tree-rewriting/type-inference syntax, e.g. `{infer Type}`) is the one construct explicitly identified, but not yet implemented, as the next candidate for extension. Mixed `Alternatives` (keywords mixed with rule calls, or list-assignments to more than one shape) fall back to visiting only the first branch, emitting a warning rather than failing outright, so the pipeline always produces *something*. Cross-reference scoping and unordered-group ordering choice, discussed above, are both deliberate simplifications rather than oversights, each documented at the point in the code where the simplification is made.

5 Evaluation

5.1 Validation

Correctness is checked by 45 tests across `tests/validation/`, run with Node's built-in test runner (no external test framework dependency): one file per pipeline module (`grammar-loader`, `validator`, `ir-builder`, `block-json-generator`, `blockly-ts-target`), plus `roundtrip.test.js`, which *compiles* the generated `blocks.ts`/`generator.ts` to real ES modules, loads them against the real `blockly` npm package, builds an actual headless `Blockly.Workspace`, wires up blocks the way a person dragging them in the UI would, and asserts on the DSL text Blockly's own generator reconstructs — this is possible without a browser because Blockly's model (workspace, blocks, connections, fields) has no DOM dependency, only its SVG renderer does.

A separate 12-test suite in `tests/evaluation/` checks breadth rather than individual correctness: that every bundled example grammar (and negative fixture) produces the outcome documented for it — pipeline success, a load-time syntax/linking error, or a validate-time rejection — that every one of the six documented `IRPart.kind` values is exercised by at least one bundled grammar, and that every construct the supported-subset table names as unsupported (currently `Action`) is in fact rejected. All 45 validation tests and all 12 coverage tests pass on the current implementation, including the `UnorderedGroup` extension from Section 4.4.

5.2 Performance Cost

We benchmarked each pipeline stage against synthetic grammars of increasing size (1 to 200 rules, 7 timed repetitions after 2 warm-up runs, single laptop), summarised in Table 1.

Table 1. Median stage time (ms) vs. grammar size

Rules	Load	Validate	Build IR	Generate	Total
1	21.39	0.00	0.00	0.04	21.44
5	25.34	0.01	0.01	0.15	25.51
10	25.55	0.02	0.02	0.28	25.88
25	38.23	0.05	0.06	0.75	39.09
50	53.02	0.05	0.12	1.50	54.69
100	109.92	0.31	0.23	2.35	112.81
200	163.23	0.40	0.40	5.03	169.06

From 1 to 200 rules (a 200× increase, if scaling were perfectly linear), measured time grows by 7.6× for loading, 111.8× for validation, 89.7× for IR construction, and 126.8× for code generation — i.e. validation, IR construction, and generation all scale sub-linearly with grammar size at this range, while

`loadGrammar` is dominated by a largely size-independent fixed cost: it calls `createLangiumGrammarServices()` fresh on every invocation. This is immaterial for the pipeline's intended one-shot CLI usage, but is worth caching if the pipeline were ever driven from a long-running process (e.g. a watch mode) converting many grammars per process lifetime — a limitation we document in the code rather than treat as a surprise.

6 Discussion & Conclusion

We implemented a four-stage pipeline that turns a restricted-subset Langium grammar into a working Blockly editor, validated it against a 45-test unit/integration suite and a 12-test breadth-coverage suite (all passing), and benchmarked it up to 200-rule synthetic grammars with sub-linear scaling on every stage but grammar loading. Beyond the base pipeline, we used two features — cross-references rendered as a live, workspace-scanning dropdown, and Langium's unordered-group operator — as concrete case studies in extending the pipeline without destabilising it, in both cases following (and, for `UnorderedGroup`, directly applying) the three-step extension pattern the codebase documents: extend the validator's allow-list, add an IR-builder handler, and only touch code generation if a genuinely new `IRPart.kind` is required.

The clearest direction for future work is closing the one remaining gap the supported-subset table names explicitly: `Action` support, which would additionally require deciding how Blockly should represent Langium's type-inference/tree-rewriting semantics — a substantially different kind of construct from the five we currently support, since it does not correspond to a single, obvious block input the way an assignment or a keyword does. Scoped cross-references (restricting a reference's live-scanned options to blocks actually reachable from the referencing block's position on the workspace, rather than every declared instance workspace-wide) and an "optional input" mutator for `?`-cardinality fields are two further, smaller extensions the current architecture is already well positioned to support.

References

1. Google: Blockly — a library for building block editors. `https://developers.google.com/blockly`, accessed September 2026
2. TypeFox: Langium — the language engineering tool. `https://langium.org/`, accessed September 2026